

LLMOrchestrator

LLMOrchestrator

Единая управляющая плоскость для всех бессеансовых CLI-агентов кодинга.

Краткое описание

LLMOrchestrator — это автономный, переиспользуемый Go-модуль для запуска, управления и взаимодействия с бессеансовыми CLI-агентами (OpenCode, Claude Code, Gemini, Junie, Qwen Code) по гибриднему каналу «пайп+файл», с индивидуальными предохранителями для каждого агента, подключаемым механизмом выбора провайдеров, развязанной абстракцией интернационализации и гарантиями защиты от ошибочных тестов.

Краткая аннотация

Переиспользуемый Go-модуль, предоставляющий единый интерфейс для запуска и управления несколькими CLI-агентами на базе LLM через гибридный протокол «пайп+файл». Потокбезопасный пул агентов с предохранителями и настраиваемыми стратегиями маршрутизации, сознательно независимый от потребителя, с подключаемым переводчиком для интернационализации.

Подробное описание

LLMOrchestrator — это общая инфраструктура для оркестрации бессеансовых CLI-агентов кодинга, та самая «подводка», которая необходима любой мультиагентной системе, но которую обычно реализуют кое-как. Вместо того чтобы каждый проект заново изобретал запуск процессов, форматирование сообщений и парсинг результатов для таких инструментов, как OpenCode, Claude Code, Gemini CLI, Junie и Qwen Code, модуль предлагает единый интерфейс Agent,

потокобезопасный AgentPool и MultiProviderPool, объединяющий агентов от разных провайдеров за единым фасадом. Маршрутизация настраивается через AgentSelector — это может быть циклический выбор с пропуском провайдеров, не соответствующих требованиям, или приоритетный с резервными вариантами. Таким образом, распределение задач становится вопросом политики, а не жёстко закодированного допущения. Каждый конкретный агент — это тонкая обёртка над общим BaseAdapter, который управляет всем жизненным циклом процесса: запуск с настройкой пайпа, корректная остановка через SIGTERM, затем SIGKILL, перезапуск и проверка работоспособности — все эти сложные и подверженные ошибкам моменты решены раз и навсегда.

Взаимодействие намеренно гибридное, чтобы транспорт соответствовал задаче. Пайп-транспорт передаёт JSON-сообщения с разделителями в виде символов новой строки, с тайм-аутом на чтение каждого запроса и ограничением длины ответа для быстрого интерактивного обмена, а файловый транспорт использует отдельные каталоги входящих/исходящих/общих данных для каждой сессии, чтобы хранить крупные или долговременные артефакты, которые не должны попадать в пайп. Устойчивость к сбоям заложена в архитектуру, а не добавлена постфактум: индивидуальный предохранитель для каждого агента срабатывает после трёх последовательных ошибок, переводя его в режим охлаждения на 60 секунд, после чего следует пробный запрос в полуоткрытом состоянии. Фоновый монитор работоспособности периодически проверяет агентов, позволяя восстановить упавший агент ещё до того, как его отсутствие заметит входящий трафик. Захват агента из пула блокируется на условной переменной, а не расходует процессорное время в бесконечном цикле ожидания, а парсер ответов не имеет состояния и безопасен для параллельного вызова. Модуль строго развязан — в него не просачиваются специфичные для потребителя детали — и все пользовательские строки проходят через подключаемый интерфейс интернационализации Translator, где NoopTranslator возвращает идентификаторы сообщений в исходном виде, чтобы отсутствие перевода бросалось в глаза, а не маскировалось.

Зачем мы его создали

Любая мультиагентная система должна надёжно запускать CLI-агентов и взаимодействовать с ними. Каждый раз заново решать вопросы запуска, форматирования, парсинга и обработки ошибок — это расточительно и чревато ошибками. LLMOrchestrator централизует эти задачи в одном развязанном, переиспользуемом модуле, чья узкая

специализация делает его универсальным — и эта универсальность разрушается, как только в него проникают специфичные для потребителя детали.

Почему это меняет правила игры

Это превращает задачу «управлять армией разнородных агентов CLI» из уникальной инженерной рутины для каждого проекта в простой импорт библиотеки — пул соединений, размыкатель цепи, управление жизненным циклом и подключаемая маршрутизация уже решены и проверены на практике. А благодаря антиблефовым тестам, которые проверяют реальную систему от начала до конца, а не довольствуются «компилируется», вы получаете абстракцию, которой можно доверять в условиях конкуренции и сбоев, а не просто красивую схему.

Что нового

- **Гибридный протокол «канал+файл»** — интерактивная скорость (строки JSON через stdin/stdout, тайм-ауты чтения, ограничения на размер ответов) и надёжный файлообмен (inbox/outbox/shared) для крупных артефактов, так что вам никогда не придётся жертвовать задержкой ради надёжности или наоборот.
- **Пул с несколькими провайдерами и подключаемыми селекторами** — единый фасад для множества провайдеров CLI, где маршрутизация по круговому алгоритму или с учётом приоритетов выбирается как политика, а не защита намертво.
- **Размыкатель цепи для каждого агента + фоновый мониторинг работоспособности** — автоматическое снижение нагрузки и восстановление (3 сбоя → 60 секунд в открытом состоянии → пробный полуоткрытый режим), так что нестабильный агент изолируется, а затем тихо возвращается в строй без ручного вмешательства.
- **Пул без активного ожидания** — Acquire блокируется на `sync.Cond`, пока не освободится подходящий работоспособный агент или не отменится контекст, так что ожидание не расходует процессорное время.
- **Жёсткое разделение + антиблефовая i18n** — NoopTranslator возвращает идентификаторы сообщений в исходном виде, так что отсутствие перевода невозможно пропустить — вместо тихого вывода пустой строки.
- **Безопасность по умолчанию** — белый список путей к бинарным файлам исключает интерполяцию командной оболочки и, следовательно, поверхность для инъекций, подкреплён защитой от обхода пути, ограничением

размера ответа в 1 МиБ против неконтролируемого вывода и маскировкой ключей API в логах.

- **Антиблефовый тестовый фреймворк Challenge** — реальные сквозные проверки на диске, через JSON и парсер в пяти локалях, с парным мутационным шлюзом, который должен завершаться с ненулевым кодом, если функция сломана — тест, доказывающий, что система действительно может дать сбой.

Крупнейшие технические вызовы и их решения

- **Надёжный ввод-вывод процессов агентов.** Общение с запущенным процессом CLI обманчиво сложно; решено с помощью гибридного транспорта «канал+файл», чёткого контракта сообщений и парсера, чтобы обе стороны согласовывали формат передачи, и BaseAdapter, централизующего весь жизненный цикл процесса, включая корректное завершение по SIGTERM с эскалацией до SIGKILL в крайнем случае.
- **Конкуренция без активного ожидания.** Решено с помощью AgentPool на мьютексе и условной переменной, где Acquire «засыпает», пока не освободится агент с нужными возможностями, в сочетании с потокобезопасным парсером без побочных эффектов, который можно безопасно вызывать из множества горутин одновременно.
- **Изоляция сбоев провайдеров.** Решено так, что один неисправный провайдер не тянет за собой остальных: размыкатели цепи на уровне агентов ограничивают радиус поражения, а фоновая горутин мониторинга работоспособности обеспечивает восстановление, даже если запросов нет.
- **Доказательство корректности, а не просто компилируемости.** Решено с помощью фреймворка Challenge: десятки инвариантов на пяти языках (en/sr/ja/es/de), проверяющих реальную систему, плюс парный мутационный шлюз (LLMORCH_MUTATE_RUNNER=1 должен завершаться с ошибкой → обёртка возвращает код 99), который намеренно ломает функцию, чтобы доказать, что сам шлюз не блефует.
- **Локализация без скрытых сбоев.** Решено с помощью шва NoopTranslator, возвращающего идентификаторы в исходном виде, и инъекции переводчика для каждого потребителя, так что пробелы в переводах всегда заметны, а не замаскированы.

Технологический стек

- **Go (1.25)** — выбран за первоклассную поддержку конкурентности и чёткое управление процессами, что критически важно для оркестрации живых агентских процессов; реализует модуль, адаптеры агентов, транспорты и парсер.
- **Go только из стандартной библиотеки (+ testify, yaml.v3)** — осознанный выбор, чтобы минимизировать поверхность зависимостей и не подтягивать *никакие* SDK LLM, благодаря чему модуль остаётся лёгким и может быть встроен в любой потребительский проект без привнесения стороннего багажа.
- **Транспорт на основе каналов (JSON-lines поверх stdio)** — выбран для быстрого интерактивного обмена сообщениями, усиленного тайм-аутами на чтение и ограничениями на длину ответов, чтобы зависший или вышедший из-под контроля агент не мог заблокировать вызывающую сторону.
- **Файловый транспорт (inbox/outbox/shared)** — выбран для надёжного обмена крупными артефактами в рамках сессии, где каналы были бы неподходящим инструментом.
- **sync.Mutex/sync.Cond** — выбраны для реализации блокирующего, справедливого доступа к пулу агентов без активного ожидания.
- **Автоматический выключатель + HealthMonitor** — выбраны в связке, чтобы обеспечить устойчивость на уровне каждого агента и активное восстановление, а не просто обнаружение сбоев.
- **Переводчик из pkg/i18n** — выбран как независимый слой локализации, отделяющий специфичные для потребителя строки от ядра.
- **Тестовый фреймворк (challenges/runner) + Makefile (test -race, fuzz, cover)** — выбраны для проверки без самообмана, подкреплённой доказательствами, включая обнаружение состояний гонки и фаззинг парсера, чтобы корректность подтверждалась в условиях искусственно созданных помех, а не просто постулировалась.

Статус и замечания о честности

- **Статус: бета-версия.** Автономный многократно используемый модуль, подключаемый как submodule в нескольких проектах Helix/vasic. **Лицензия: Apache-2.0;** репозиторий GitHub открыт.
- Метаданные моделей поступают из LLMsVerifier через мост HelixQA; этот модуль не импортирует LLMsVerifier/VisionEngine/DocProcessor напрямую. Стеки, упоминаемые

в CLAUDE.md родительского приложения (Gin/PostgreSQL и т. д.), описывают helix_code, а не этот модуль.

Приоритетный уровень: Helix-первичный (инфраструктурный кластер LLM — автономный многократно используемый модуль). Уступает по приоритету HelixTrack.